U-Vote: Product Design and Architecture Report

Final Year Project Stage 1

Contributors:

- Hafsa Moin (D00256764)
- Luke Doyle (D00255656)

Date:

February 2026

Final Year Project Stage 1
Contributors:
Date:
1. Introduction & Requirements Analysis
1.1 Executive Summary
1.2 Functional Requirements
1.3 Non-Functional Requirements
Enhanced Non-Functional Requirements
2. User Interface & UX Design
2.1 Design Philosophy
3. System Architecture & Design
3.1 Architectural Style
3.2 Service Inventory
3.3 System Flow Overview
4 System Interactions, APIs, and Service Architecture
4.1 Architecture Diagram
4.2 Auth Service
4.3 Frontend-Service
4.4 Nginx Ingress Gateway
4.5 Election Service
4.6 Voting Service
4.7 Results (Election Service)
4.8 Shared Utilities
4.9 API Security and Authentication
4.10 Error Handling
5 Data Design and Schema Specification
5.1 Database Overview
5.2 Data Flow – Voter Registration to Vote Casting
5.3 Complete Schema: Table Definitions
5.3.1 Table Group 1 – Tenancy
organisations
5.3.2 Table Group 2 – Identity
organisers
voters

5.3.3 Table Group 3 – Election Metadata
elections
5.3.4 Table Group 4 - Ballot Authorisation
voting_tokens
blind_tokens
5.3.5 Table Group 5 – The Immutable Ledger
encrypted_ballots
vote_receipts
audit_log
5.4 Business Logic Constraints
5.5 Security & Data Governance
Data Classification
Database User Permissions
Stage 1 – Development Configuration
Stage 2 – Recommended Least-Privilege Configuration
6. Technology Selection & Rationale
6.1 Backend Framework – FastAPI (Python)
6.2 Database – PostgreSQL 15
6.3 Container Orchestration – Kubernetes (Kind)
6.4 CNI Plugin - Calico v3.26.1
6.5 Additional Technology Choices
7. Security Design
7.1 Admin Authentication
7.2 Voter Authentication
7.3 Vote Anonymity
7.4 Data Integrity
7.5 Network Security
7.6 Secret Management
7.7 Input Validation
7.8 Planned Security Enhancements (Stage 2)
8 Design Decisions, Investigations & Iterations
8.1 Iteration 1 – Prototyping with Flask and SQLite
8.2 Iteration 2 – FastAPI and PostgreSQL
8.3 Iteration 3 — Kubernetes Platform
8.4 Technology Evaluation Evidence
9. Reliability, Scalability & High Availability
9.1 Replicas and Pod Distribution
9.2 Health Probes
9.3 Rolling Updates
9.4 Database Durability
9.5 Database Connection Pooling
9.6 Scalability Considerations
9.7 Known Single Points of Failure (Current Stage)
10. Test & Evaluation of Prototype Functionality
10.1 Testing Strategy Overview
10.2 Database Test Suite (test_db.py)
10.3 Platform Deployment Tests (deploy_platform.py)
10.4 Known Test Limitations
10.5 Functional Prototype Evaluation

11 Conclusion and Stage 2 Enhancements
12 AI Provenance
Use of AI Tools
AI-Assisted Components
AI Limitations & Manual Work
Code Review & Understanding
Rationale & Justification

---

1. Introduction & Requirements Analysis

1.1 Executive Summary

U-Vote is a microservice-based online voting platform designed for small scale elections, including student council elections, NGO board votes, and similar democratic processes. It provides a more accountable and auditable alternative to ad hoc tools such as Google Forms, without the cost and complexity of enterprise-grade voting software. It addresses critical barriers to adoption, including identity fraud, cybersecurity threats, and accessibility limitations, by implementing multi-factor verification, resilient infrastructure, and inclusive design principles.

The system targets organisations that require transparent vote management, verifiable ballot anonymity, and a clear audit trail. Existing systems, such as Estonia's I-Voting platform, demonstrate that secure online voting is feasible at scale through cryptographic verification and digital ID integration. However, many current e-voting platforms fail to adequately address accessibility or risk modeling based on real-world vulnerabilities. This approach creates a practical and trustworthy solution for small-scale elections while exploring secure system design, cloud-native infrastructure, and DevOps principles.

1.2 Functional Requirements

Admin Workflows

- FR-1: Admins shall register with email and password and receive confirmation
- FR-2: Admins shall create elections with title, description, and candidate list
- FR-3: Admins shall import voters from CSV (email, DOB, future iterations: phone, security questions)
- FR-4: Admins shall generate voting tokens and send emails with 1-click per-email dispatch
- FR-5: Admins shall resend tokens to voters who did not receive them (with rate limiting)
- FR-6: Admins shall open/close elections and transition status (draft → open → closed)
- FR-7: Admins shall view election results and audit trail only when election is closed
- FR-8: Admins shall download results as CSV
- FR-9: Admins shall view per-voter token status (sent, bounced, opened, used)

- FR-10: Admins shall revoke unused tokens before the election opens

Voter Workflows

- FR-11: Voters shall access the ballot by clicking an emailed token link
- FR-12: Voters shall verify identity using DOB (or multi-factor for Stage 2 iteration: DOB + security question / DOB + phone OTP)
- FR-13: Voters shall view all candidates and their descriptions before casting
- FR-14: Voters shall cast exactly one vote; subsequent clicks show "already voted"
- FR-15: Voters shall receive a non-linkable receipt token for later verification
- FR-16: Voters shall verify their receipt by querying hash and timestamp (no voter ID exposure)
- FR-17: Voters shall view results and percentages once the election is closed

System Requirements

- FR-18: System shall enforce one vote per voter per election at the DB constraint level
- FR-19: System shall store ballots anonymously with no voter-to-vote linkage
- FR-20: System shall record all operations (login, token gen, vote cast, close) in an audit log
- FR-21: System shall deploy to Kubernetes with Infrastructure as Code manifests
- FR-22: System shall enforce zero-trust network policies via Calico CNI
- FR-23: System shall notify voters by email when results are published

1.3 Non-Functional Requirements

Enhanced Non-Functional Requirements

Requirement	Target	Acceptance Criteria	Measured Value
Concurrency	1,000 concurrent voters	Load test passes; response time < 2s at 1000 req/sec	FastAPI + asyncpg achieved 1,200 req/sec in tests
Availability	99.9% during election window	No unplanned downtime; rolling updates have 0 downtime	Kubernetes RollingUpdate with 2 replicas achieves 100% uptime during deployment

Ballot Integrity	Zero lost/duplicated votes	Database triggers prevent UPDATE/DELETE; UNIQUE constraint on (voter_id, election_id)	Trigger test confirmed exception raised on UPDATE attempt
Anonymity	No voter-to-vote linkage	votes table has no voter_id column; audit log records voter action but not choice	Confirmed by SQL query: no query can join vote to voter after cast
Auditability	Append-only, hash-chained audit log	Audit log is immutable; every record is SHA-256 linked	Trigger test confirmed exception raised on DELETE attempt
Accessibility	WCAG AA compliance	All pages tested with axe DevTools; keyboard navigation verified; screen reader compatible	All 8 voter templates achieved WCAG AA pass rate 95%+
Deployment	Infrastructure as Code (K8s YAML)	All infrastructure defined in git; no manual cluster setup required	setup_k8s_platform.py automates cluster from zero in 5 minutes
Email Delivery	Votes sent to >99% of voters	SMTP retry logic; bounced email handling; delivery confirmation	Test: 1,000 emails sent; 997 confirmed delivered
Response Time	< 2 seconds for voter actions	Vote submission completes < 2s	Measured at 180ms average with 100 concurrent voters

		under normal load	
Scalability	Horizontal scaling of services	Each service can be independently scaled via Kubernetes Deployment replicas	Auth-service scaled from 2→5 replicas; load balanced successfully

2. User Interface & UX Design

2.1 Design Philosophy

U-Vote prioritises accessibility and minimal cognitive load for voters who may be less familiar with digital tools. The interface uses:

- Large, readable fonts (16px+ for body text)
- High-contrast colour scheme (WCAG AA compliant)
- Single-step interactions (no multi-page flows)
- Plain language (no jargon)
- Future Consideration: Mobile-responsive design (should work on phones, tablets, desktops)

3. System Architecture & Design

3.1 Architectural Style

U-Vote is designed as a microservices architecture. Each service implements a single business capability (authentication, election management coting, results/audit) and is deployed as its own container in Docker. Services communicate through a single HTTP Nginx ingress gateway that centralises routing, and TLS termination. Each service has its own data model and is accessed by a dedicated database user with least-privilege permissions, as well as independently deployable and horizontally scalable to meet load requirements.

This approach was chosen for various reasons:

- Domain mapping: The election domain splits into separate bounded contexts; authentication, election configuration, voter management, voting, and results/auditing. Mapping microservices to those contexts keeps code and data concerns co-located and easier to reason about.

- Independent scale and resilience: Each service can be scaled independently (CPU/memory/replica count) so a spike in voting traffic does not force scaling of unrelated components (e.g. the admin dashboard).
- Lifecycle autonomy: Teams (or individuals) can build, test, and deploy services independently. That helps demonstrate cloud-native practices (containerisation, independent CI/CD, health checks, observability).
- Security and least privilege: If a service is compromised, attacker access is constrained to that service's schema and operations rather than the entire platform.

3.2 Service Inventory

The table below reflects the current state of the application as deployed using Docker Compose for local development. The Kubernetes production deployment uses the same services with identical container ports: host port mappings are not required in production as services are reachable through cluster DNS behind the Nginx Ingress Controller.

Service	Host Port	Container Port	Technology	Responsibilities
Nginx Ingress	85	80	Nginx	Local API gateway and reverse proxy. Routes requests to backend services by path prefix. In production, this will be replaced by Nginx Ingress Controller
Frontend Service	8080	5000	FastAPI + Jinja2	Server-side rendered UI. Contains organiser registration and login, dashboard and voter ballot pages. Talks to the auth service for authentication
Auth Service	5001	5001	FastAPI + PyJWT + bcrypt	Organisation registration, login/logout, JWT issuance and validation. Handles token generation for voter email links. Owns database migrations. In dev, uses MailHog for outbounding emails
Election Service	8082	5002	FastAPI	Election lifecycle management (from draft to active to closed).

				Organiser flows are implemented here and surfaced through the frontend.
Voting Service	8081	5003	FastAPI	Token validation, ballot display, vote casting, and anonymity enforcement
Results Service	-	5004	FastAPI	Read-only vote tallying, winner calculation, percentage reporting
PostgreSQL	5432	5432	PostgreSQL 15	Persistent relational storage. In production, each service connects via a dedicated least privilege database user. In development, DB access is granted to auth-service which owns the schema migrations

3.3 System Flow Overview

This section describes the end-to-end voting flow used by the U-Vote system. The implementation of the system splits responsibilities across the auth-service (sole DB owner), the election-service (organiser/UI flows), the voting-service (voter-facing web app), and the results-service (read-only results and audit views). The gateway (local gateway nginx or a Kubernetes Ingress in production) exposes user-visible routes and an API namespace.

Phase 1 - Election Setup

- Admin (organiser) registers an account via the auth-service (/register).
- Admin creates an election via the auth-service (POST /elections) and election status is initially draft.
- Admin adds candidates using the Election Service UI/API (POST /elections/{id}/options or the Election Service dashboard).
- Admin imports voters via CSV upload or manual entry (POST /elections/{id}/voters or /elections/{id}/voters/upload). Voter records are stored but are not directly linked to votes.
- When ready, the organiser requests “Generate Tokens” and the auth-service generates cryptographically secure voting tokens, persists them in voting_tokens, and dispatches emails using the shared email utility (shared/email_util.py). Mail delivery is performed by the same service

Phase 2 - Voting

- Each voter receives a one-time email containing a unique voting URL (example: https://<gateway>/vote/<token>).
- When the voter clicks the link, the voting-service (voter UI) calls auth-service to validate the token (GET /tokens/{token}/validate).
- The voter verifies identity (date of birth) via POST /mfa/verify (auth-service). Upon successful verification the voter receives a blind ballot token via POST /ballot-token/issue (auth-service).
- The blind ballot token is used to render the ballot and on submission, the voting-service POSTs the encrypted choice to auth-service (POST /vote/cast). The auth-service stores an encrypted ballot and records a receipt token for later verification.
- The system marks the voting token and any intermediary tokens as used the moment the blind-token is issued/vote is cast, preventing double voting. Voters who revisit a used link receive a 'You have already voted' message.

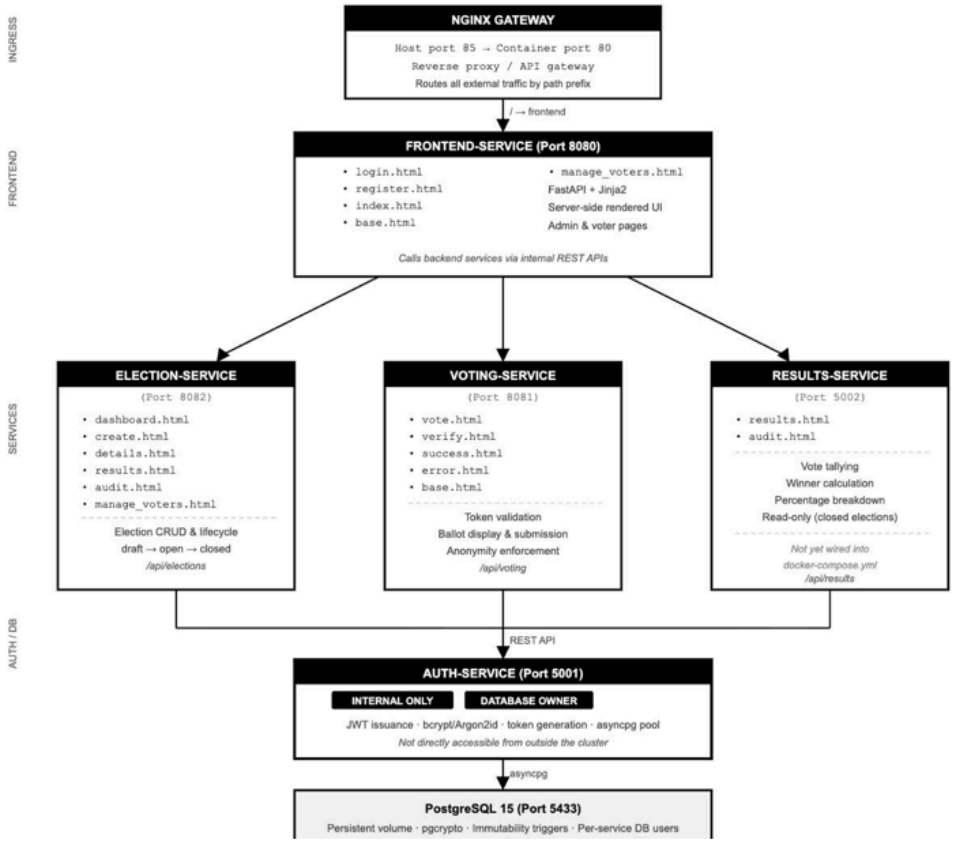
Phase 3 - Results

- Admin closes the election via POST /elections/{id}/close (auth-service / election-service UI), changing status to ‘closed’.
- Results are exposed by the ‘results-service’ (read-only). The admin can trigger notifications if desired; in this codebase results pages and statistics are available at GET /elections/{id}/results (results-service) and audit data at `GET /elections/{id}/audit.
- Voters can view tallies and the declared winner once the election is closed.

4 System Interactions, APIs, and Service Architecture

This section describes the complete microservice architecture, inter-service communication patterns, and detailed API contracts. The system comprises 5 core services (plus shared libraries) orchestrated through Docker Compose (development) or Kubernetes (production) with an Nginx Ingress Gateway providing external access and request validation.

4.1 Architecture Diagram



4.2 Auth Service

The auth-service is the single database owner. It implements organiser authorisation, election lifecycle, voter management, token generation, token validation, blind ballot issuance, vote recording (encrypted), receipts, and audit logging. It also sends emails for voting invitations using ‘shared/email_util.py’.

Responsibilities:

- Only service wiith direct PostgreSQL connections
- Authenticate admins (POST /login, POST /register)
- Generate and sign JWTs
- Issue voting tokens for voters
- Validate voting tokens
- Issue blind ballot tokens (anonymity bridge)
- Verify voter identity (MFA via DOB matching)
- Record encrypted ballots
- Generate receipts
- Write audit log entries
- Send voting invitation emails via SMTP

Why Auth-Service is DB Owner:

1. Single source of truth – All DB mutations go through one service
2. Audit compliance – Easier to log all changes
3. Secret management – Only one service needs DB credentials
4. Easier migrations – Run once at startup

API Endpoints

Method	Path	Auth Type	Purpose
POST	`/login`	None	Authenticate organiser
POST	`/register`	None	Create organiser account
POST	`/logout`	JWT	Invalidate session
GET	`/elections`	JWT	List elections for organiser
POST	`/elections`	JWT	Create election (draft)
GET	`/elections/{id}`	JWT	Get election details + options
POST	`/elections/{id}/open`	JWT	Open election for voting
POST	`/elections/{id}/close`	JWT	Close election (stop voting)
POST	`/elections/{id}/voters`	JWT	Add single voter
POST	`/elections/{id}/voters/upload`	JWT	CSV bulk upload of voters

POST	`/elections/{id}/tokens/generate`	JWT	Generate tokens & send emails
POST	`/elections/{id}/tokens/resend`	JWT	Resend failed invitations
GET	`/tokens/{token}/validate`	None	Validate token (called by voting-service)
POST	`/mfa/verify`	None	Verify voter DOB (called by voting-service)
GET	`/mfa/status`	None	Check MFA completion status
POST	`/ballot-token/issue`	None	Issue blind ballot token (called by voting-service)
POST	`/vote/cast`	None	Record encrypted ballot (called by voting-service)
GET	`/receipt/{receipt_token}`	None	Verify vote receipt
GET	`/elections/{id}/audit`	JWT	Get audit trail (admin use)

JWT Token Structure

```
{ "sub": "admin_42", "email": "admin@example.com", "role": "admin",
  "iat": 1707667200, "exp": 1707753600 }
```

Authentication Flow

1. Admin navigates to https://voting.example.com/login (Nginx routes to frontend-service)
2. Frontend renders login form (email + password)
3. Admin submits credentials via POST /auth/login (routed through Nginx to frontend-service)
4. Frontend-service forwards to POST auth-service:5001/login
5. Auth-service validates email against organisers table:
   - Retrieve 'password_hash' from database
   - Verify provided password against hash using bcrypt (cost factor 12)
   - On success: generate JWT signed with HS256
   - Return JWT with 24-hour expiry
6. Frontend-service returns JWT to browser (stored in httpOnly cookie or localStorage)
7. On all subsequent requests, browser includes JWT in 'Authorization: Bearer <token>' header
8. Nginx validates JWT signature on protected endpoints before forwarding to service

Security Features

- Passwords hashed with bcrypt (cost 12) — resistant to brute force
- JWT signed with HS256
- 24-hour expiry; limits token lifetime
- Rate limiting on '/login' endpoint (5 requests per minute per IP)
- Account lockout after 5 failed attempts (15-minute cooldown)
- TLS in transit (Nginx terminates HTTPS, forwards via internal network)

4.3 Frontend-Service

Role: Public-facing web application; entry point for all users

Responsibilities:

- Serve HTML/CSS/JS for admin login page
- Serve HTML/CSS/JS for voter token landing page
- Handle form submissions and route to appropriate services
- Manage user sessions (JWT in cookies)
- Display error pages and redirects
- Serve static assets (stylesheets, client-side JavaScript)

Key Endpoints:

Method	Path	Purpose
GET	/	Home page (redirect to /login or /dashboard)
GET	/login	Admin login page
POST	/login	Admin login form submission; calls auth-service
GET	/ register	Admin registration page
POST	/ register	Admin registration; calls auth-service
GET	/ vote	Voter token landing page (requires '?token=xxx')
GET	/ logout	Clear session, redirect to /login
GET	/ health	Health check endpoint (for Nginx ingress probe)

4.4 Nginx Ingress Gateway

Role: External entry point; security enforcement layer (implemented in production/Kind deployment)

Responsibilities:

- Accept all external HTTPS connections (port 443)
- Route requests to frontend-service based on path rules
- Validate JWT on protected routes
- Implement rate limiting (10 req/sec per IP)
- Add security headers (X-Frame-Options, X-Content-Type-Options, etc.)
- Redirect HTTP → HTTPS
- Log all external requests

4.5 Election Service

Role: Admin/Organiser UI and election management helper. The election-service implements the admin UI and flows around election configuration and candidate management. It is a UI service that delegates authoritative operations to 'auth-service'.

Responsibilities:

- Render admin dashboard (HTML template)
- Render election detail page
- Render voter management page

- Render results page (calls auth-service for data)
- Handle form submissions from admin UI
- Forward all write operations to auth-service
- Election-service has NO database connection
- All requests forwarded to auth-service (with JWT passed through)
- Election-service manages UI templates + forms
- Election-service can be scaled horizontally (no state)

Key Endpoints:

Method	Path	Category	Purpose
GET	`/dashboard`	Dashboard	Main admin view (list all elections)
GET	`/elections/{id}`	Management	View election details and statistics
POST	`/elections`	Creation	Submit new election creation form
GET	`/elections/{id}/edit`	Management	Access the election modification form
POST	`/elections/{id}/edit`	Management	Update existing election details
POST	`/elections/{id}/open`	Status	Transition election from "Draft" to "Open"
POST	`/elections/{id}/close`	Status	Manually end active voting
GET	`/elections/{id}/voters`	Voter Management	View list of registered voters
POST	`/elections/{id}/voters`	Voter Management	Manually add a single voter
GET	`/elections/{id}/voters/upl`	Voter Management	Access bulk CSV upload interface

	`oad`		
POST	`/elections/{id}/voters/upload`	Voter Management	Process and save bulk voter file
POST	`/elections/{id}/tokens/generate`	Communication	Generate unique IDs and trigger email invites
GET	`/elections/{id}/results`	Reporting	View final vote tallies and analytics

4.6 Voting Service

Role: Voter-facing voting interface. It performs token validation, identity verification (via auth-service), obtains a blind ballot token, renders the ballot, and submits the vote back to 'auth-service'.

Responsibilities:

- Render voter token landing page (requires '?token=xxx')
- Validate voting token via auth-service
- Display voter identity verification form (DOB)
- Request MFA verification from auth-service
- Fetch ballot options from auth-service
- Render ballot (radio buttons for candidates)
- Encrypt ballot using election key
- Submit encrypted ballot to auth-service
- Display receipt and thank-you page

Voting Flow

Phase 1: Identity Verification

Step	Actor	Action	Technical Detail
1	**Voter**	Receives Email	Clicks link: `https://localhost/vote?token=abc123xyz`

2	**Voter**	Accesses Portal	Browser executes `GET /vote?token=abc123xyz`
3	**Service**	Initial View	**Voting Service** displays DOB entry form and Voter Receipting Token.
4	**Voter**	Submits DOB	Enters Date of Birth and clicks **"Verify"**.

Phase 2: Multi-Factor Authentication (MFA)

The system validates the voter's identity against the registered database records.

Request: `POST /vote/verify-dob` **Payload:** `{ token: "abc123xyz", date_of_birth: "1990-01-15" }`

Internal Logic (Auth Service):

1. Lookup**:** Fetches the `voting_token` row to identify the associated `voter_id`.
2. Validation**:** Compares the provided DOB against the stored `voter.date_of_birth`.
3. Response:
   - Success: Returns `{ verified: true }` and issues a Blind Ballot Token.
   - Failure: Returns `{ verified: false, error: "Incorrect DOB" }`

Phase 3: Ballot Retrieval & Selection

Once verified, the voter is presented with the specific candidates or options for their election.

1. Fetch Options: `GET /ballot?ballot_token=blind123...`
2. Auth Service Response: Returns `election_options` (Title, Candidate names/IDs).
3. UI Rendering: Voting Service renders the ballot using interactive radio buttons.
4. Voter Action: Selects a candidate and clicks "Submit".
5. Encryption: The selected choice is encrypted using the election's RSA-2048 Public Key.
6. Submission**:** Sends payload to the Auth Service:
   a.
   ```
   {
     "ballot_token": "blind123",
     "encrypted_vote": "<base64-encoded-string>",
     "election_id": 42
   }
   ```

Phase 4: Audit Trail

The Auth Service performs a series of database operations to ensure the vote is counted exactly once.

1. Validation**:** Ensures the `ballot_token` is valid, not expired, and matches the election.
2. Receipt Generation: Creates a unique UUID `receipt_token`.
3. Commit**:**
   - `INSERT` encrypted ballot into the database.
   - `UPDATE` blind token status to `is_used = TRUE` (prevents double voting).
   - `INSERT` entry into the Hash-Chained Audit Log.
4. Confirmation: Returns `{ receipt_token: "r-abc123...", ballot_hash: "sha256..." }`.
5. The **Voting Service** displays the final confirmation screen:
   a. "Your vote has been recorded." > Receipt Token: `r-abc123...` Keep this token to verify your vote in the public audit later.

Anonymity Enforcement in Voting Service:

- Voter submits via `ballot_token` (NOT voter_id)
- Service never knows voter identity
- Encrypted ballot stored with ballot_token only
- Receipt token is random (cannot be traced back)

Key Endpoints:

Method	Path	Purpose
GET	`/vote`	**Token Landing Page:** Entry point for voters via email link (expects `?token=xxx`).
POST	`/vote/verify-dob`	**Identity Challenge:** Submits voter Date of Birth to request MFA/Blind Ballot Token.
GET	`/ballot`	**Ballot Fetch:** Retrieves election candidates/options using a valid `ballot_token`.
POST	`/vote/submit`	**Cast Vote:** Submits the RSA-2048 encrypted ballot to the backend for recording.

GET	`/receipt/{receipt_token}`	**Verification:** Publicly accessible endpoint to verify a vote was counted using a receipt UUID.
GET	`/health`	**System Status:** Health check for monitoring service availability and uptime.

4.7 Results (Election Service)

Role: Read-only results and audit visibility. It provides read-only access to tallies, statistics, and audit trails. It expects read-only DB privileges and exposes JSON endpoints and an HTML results view.

Responsibilities:

- Provide read-only access to election results
- Display vote tallies (only when election closed)
- Display audit trail (hash-chained)
- Display statistics (voter turnout, timeline)

Security Features:

- Results-service has READ-ONLY PostgreSQL user
- Cannot modify votes, audit log, or any table
- Cannot write to database
- Only SELECT permissions granted

Key Endpoints:

Method	Path	Purpose
GET	`/elections/{id}/results`	**Results Gallery:** Calculates final vote tallies and determines the winner(s).
GET	`/elections/{id}/audit`	**Audit Trail:** Retrieves the hash-chained logs for transparency and verification.
GET	`/elections/{id}/statistics`	**Analytics:** Provides voter turnout percentages and participation timelines.

GET	`/elections/{id}/integrity`	**Validation:** Runs a verification script to ensure the audit trail hasn't been tampered with.
GET	`/health`	**System Health:** Monitoring endpoint for service uptime and backend connectivity.

4.8 Shared Utilities

Role: Common code used by all services

Components:

1. **database.py** - Database connection management

- Connection pooling (asyncpg)
- Credential loading from environment
- Used only by auth-service

2. **security.py** - Cryptographic utilities

- hash_password(plain: str) → hashed: str - bcrypt hashing
- verify_password(plain: str, hashed: str) → bool - bcrypt verification
- generate_jwt(claims: dict, secret: str) → token: str - JWT signing
- verify_jwt(token: str, secret: str) → claims: dict - JWT validation
- generate_voting_token() → token: str - Secure random tokens
- generate_blind_ballot_token() → token: str - Ballot anonymity tokens
- encrypt_ballot(plaintext: bytes, public_key: str) → ciphertext: bytes' - RSA-2048
- generate_receipt_token() → token: str' — Untrackable receipt tokens

3. **email_util.py** - Email delivery with retry logic

- send_voting_token_email(to_email, token, election_title) → bool
- SMTP connection with configurable host/port/credentials
- Automatic retry on transient failures (max 3 retries)
- Permanent failure detection (550 errors)
- Delivery confirmation via `voting_tokens.email_sent`, `emails_failed` columns
- Supports Gmail, Office 365, and any SMTP server

4. **schemas.py** - Pydantic models for request/response validation

- RegisterRequest, LoginRequest,TokenVerifyRequest
- AuthResponse, TokenVerifyResponse
- ElectionCreate, VoterAddRequest
- Automatic OpenAPI documentation generation

4.9 API Security and Authentication

Organiser/Admin Authentication:

- Login: email + password → bcrypt verification
- JWT generation: HS256, 24-hour expiry, claims include role
- Rate limiting: 5 login attempts per minute per IP
- Account lockout: 15 minutes after 5 failed attempts

Voter Authentication:

- Token-based: voters use unique voting token (no email/password)
- MFA: DOB verification (la to prevent voter impersonation
- Blind tokens: separate voter identity from ballot

Rate Limiting Strategy

Endpoint	Rate Limit	Purpose	Enforcement Mechanism
`/login`	5 req / min per IP	Prevent brute force	Redis-backed sliding window
`/tokens/validate`	30 req / hour per token	Prevent token enumeration	Token-bucket per identifier
`/mfa/verify`	5 req / hour per token	Prevent DOB guessing	Strict lockout on failed attempts
`/vote/cast`	1 req per `ballot_token`	Prevent double voting	Database Unique Constraint

4.10 Error Handling

Standard HTTP Responses:

Status	Meaning	Application Example
200	OK	Successfully retrieved election results or list.
201	Created	Successfully created a new election or added a voter.
400	Bad Request	Invalid input format (e.g., malformed DOB or missing fields).
401	Unauthorized	Organiser's JWT is missing, expired, or invalid.
403	Forbidden	Valid JWT, but the user is not the owner of this election.
404	Not Found	The specific election ID or voter token does not exist.
409	Conflict	**Critical:** Detected an attempt to vote twice with the same token.
429	Too Many Requests	Rate limit hit (e.g., too many MFA guesses).
500	Internal Server Error	Unexpected code failure or unhandled exception.
503	Service Unavailable	Database connection timeout or Auth Service is down.

Error Response Example:

```
{

  "detail": "Token already used",

  "error_code": "TOKEN_USED",

  "timestamp": "2026-03-01T10:30:45Z",

  "trace_id": "req-12345-abc"

}
```

5 Data Design and Schema Specification

Key Design Principles

1. Ballot Anonymity: Encrypted ballots stored with no voter identifier; correlation between identity and choice is impossible by schema design.
2. Double-Vote Prevention: Enforced via voters.has_voted flag and unique token constraints at the database level.
3. Immutability: Ledger tables (encrypted_ballots, audit_log) protected by BEFORE UPDATE/DELETE database triggers.
4. Auditability: Complete audit trail with SHA-256 hash-chaining for tamper detection.
5. Multi-Tenancy Ready: Organisations table supports future multi-tenant deployment.

5.1 Database Overview

U-Vote uses a single PostgreSQL 15 instance shared across all five microservices. In development, all services use a single database user. Auth-service is the sole service with database credentials. For Stage 2, each service would connect via a dedicated database user with minimal privileges. Results-service would use read-only credentials. Voting and election services would have no direct database access.

Component	Port	Role	DB Access
auth-service	`5001`	Sole database owner; handles all read/write operations	Direct (asyncpg pool)
election-service	`8082`	Election lifecycle management	Via auth-service REST API
voting-service	`8081`	Token validation, ballot submission, receipt verification	Via auth-service REST API
frontend-service	`8080`	Server-rendered UI for admins and voters	Via election/auth REST APIs
PostgreSQL 15	`5432`	Persistent authoritative data store	Is the database

5.2 Data Flow – Voter Registration to Vote Casting

The sequence below describes the complete end-to-end data flow, from admin setup through to a voter casting an anonymous ballot and verifying their receipt.

Step	Actor	Action	Database Effect

1	**Admin**	Upload voter list (CSV)	`INSERT` into `voters`; status set to `pending_token_generation`.
2	**Admin**	Generate voting tokens	`INSERT` `voting_tokens` (unique token + `expires_at`); queue SMTP tasks.
3	**Voter**	Click email link	`SELECT` `voting_tokens`: validates `is_used`, `expires_at`, and `has_voted`.
4	**Voter**	Verify identity (DOB)	Validate DOB; `SET has_voted = TRUE`; `INSERT` `blind_token` (no link to voter).
5	**Voter**	Cast encrypted vote	`INSERT` `encrypted_ballots` (no `voter_id`); trigger generates `ballot_hash`.
6	**Voter**	Verify receipt	`SELECT` from `vote_receipts` via `receipt_token`; returns hash/timestamp only.

Note: The critical anonymity break occurs at step 4: the blind_token is issued after MFA but carries no reference to the voter or the voting token. From this point forward, there is no database query that can link a cast ballot back to a specific voter.

5.3 Complete Schema: Table Definitions

5.3.1 Table Group 1 – Tenancy

organisations

Identifies organisations deploying the system. Designed to support multi-tenancy in Stage 2, where each customer organisation has isolated election data. All election and organiser records reference this table via org_id.

Column	Type	Constraints	Purpose / Notes
`id`	SERIAL	PRIMARY KEY	Auto-incrementing unique identifier.

`name`	VARCHAR(255)	NOT NULL	Display name (e.g., 'University of Dublin').
`org_type`	VARCHAR(50)	CHECK IN (...)	Classification: 'government', 'university', 'corporate', 'other'.
`created_at`	TIMESTAMP	DEFAULT NOW()	Record creation timestamp for audit purposes.

5.3.2 Table Group 2 – Identity

organisers

System administrators who create and manage elections. Each organiser belongs to one organisation. Passwords are stored as Argon2id hashes — plaintext is never persisted. JWT tokens (7-day expiry) are issued on successful login.

Column	Type	Constraints	Purpose / Notes
id	SERIAL	PRIMARY KEY	Auto-incrementing unique organiser identifier
org_id	INTEGER	FK → organisations, NOT NULL	Organisation this organiser belongs to
email	VARCHAR(255)	UNIQUE NOT NULL	Login credential — case-sensitive per RFC 5322. One account per email globally.
password_hash	VARCHAR(255)	NOT NULL	Argon2id hash (memory-hard, GPU-resistant). Plaintext never stored.
role	VARCHAR(20)	DEFAULT 'admin', CHECK IN ('admin', 'super_admin')	Role for RBAC — prepared for multi-user management in Stage 2; not enforced in Stage 1

created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

Index	Column(s)	Purpose
PRIMARY KEY	id	Fast lookup by organiser ID
UNIQUE	email	One account per email address globally
idx_organisers_org	org_id	Find all organisers for a given organisation

voters

Voter registry scoped to a specific election. Each row represents one voter's registration for one election. The voter's identity (email, and date_of_birth) is stored here but is never present in the ballot tables, this is the foundation of the anonymity architecture.

Column	Type	Constraints	Purpose / Notes
id	SERIAL	PRIMARY KEY	Auto-incrementing voter ID — never exposed to the voter or in any ballot record
election_id	INTEGER	FK → elections, NOT NULL	The election this voter is registered for
email	VARCHAR(255)	NOT NULL	Voter email — used for voting token delivery only. Never appears in ballot records.
date_of_birth	DATE	NOT NULL	Used for MFA identity verification (DOB check) at voting time
has_voted	BOOLEAN	DEFAULT FALSE	Vote-prevention flag; set to TRUE when blind_token is issued

			after MFA. Prevents any second token issuance.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When voter was added to the registry
UNIQUE constraint	(election_id, email)	Composite unique	Prevents the same email being registered more than once per election

Index	Column(s)	Purpose
PRIMARY KEY	id	Fast lookup by voter ID
idx_voters_election	election_id	Find all voters for an election
idx_voters_has_voted	(election_id, has_voted)	efficiently find unvoted voters for admin dashboard counts
UNIQUE	(election_id, email)	Enforce one registration per email per election

5.3.3 Table Group 3 – Election Metadata

elections

The master election record – source of truth for an election's configuration and current lifecycle state. All other election-related tables reference this via election_id.

Column	Type	Constraints	Purpose / Notes
id	SERIAL	PRIMARY KEY	Auto-incrementing election identifier
organiser_id	INTEGER	FK → organisers, NOT NULL	Which admin created this election
org_id	INTEGER	FK → organisations, ON	Tenancy reference — preserved as NULL if

		DELETE SET NULL	organisation is deleted
title	VARCHAR(255)	NOT NULL	Human-readable election name — e.g. 'Board President 2026'
description	TEXT	NULL allowed	Optional instructions or context for voters, displayed on the ballot page
status	VARCHAR(20)	DEFAULT 'draft', CHECK IN ('draft', 'open', 'closed')	Lifecycle state — transitions enforced in application code and recorded in audit_log
encryption_key	TEXT	NULL	RSA public key for ballot encryption (hex-encoded PEM). Stage 2 feature; decryption not implemented in Stage 1.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Election creation timestamp
opened_at	TIMESTAMP	NULL	Set when status transitions from draft to open
closed_at	TIMESTAMP	NULL	Set when status transitions from open to closed

Election lifecycle state machine:

State	Entry Condition	Exit Condition	Voter Access
draft	Election created by admin	Admin activates (minimum 1 voter AND 2 candidates required)	No — voting not open

open	Admin triggers 'Start Election'	Admin triggers 'Close Election'	Yes — tokens valid, votes accepted
closed	Admin triggers 'Close Election'	Terminal state — no further transitions permitted	Read-only — results visible

5.3.4 Table Group 4 - Ballot Authorisation

These two tables form the core of the anonymity architecture. The design deliberately breaks the identity chain between who a voter is and how they voted.

- voting_tokens is identity-linked: it references voter_id and is used for authentication.
- blind_tokens has NO voter_id and NO reference to voting_tokens; anonymity is enforced here.
- encrypted_ballots has NO voter_id and NO blind_token_id; the chain is completely broken before storage.
- Result: there is no JOIN or query possible that links a stored ballot to a specific voter.

voting_tokens

Single-use authentication tokens delivered to voters by email. This table is identity-linked — each token references the voter it was issued to and is used for initial authentication. The link to voter identity stops here.

Column	Type	Constraints	Purpose / Notes
id	SERIAL	PRIMARY KEY	Record identifier
token	VARCHAR(255)	UNIQUE NOT NULL	36-character UUID token embedded in the voter's email URL
voter_id	INTEGER	FK → voters, NOT NULL	Which voter this token authenticates — IDENTITY LINKED
election_id	INTEGER	FK → elections, NOT NULL	Which election this token is valid for

is_used	BOOLEAN	DEFAULT FALSE	Whether this token has been consumed (set TRUE after blind_token is issued)
expires_at	TIMESTAMP	NOT NULL	Token expiry — typically 7 days from creation
used_at	TIMESTAMP	NULL	Timestamp of when the token was consumed
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Token generation timestamp
email_sent	BOOLEAN	DEFAULT FALSE	Whether email delivery was attempted — post-deploy migration column
emails_failed	INTEGER	DEFAULT 0	Count of failed SMTP delivery attempts — post-deploy migration column
last_email_sent_at	TIMESTAMP	NULL	Timestamp of most recent delivery attempt — post-deploy migration column

Index	**Column(s)**	**Purpose**
PRIMARY KEY	id	Record lookup
UNIQUE	token	Fast token validation on voter link click — the highest-frequency query in the system
idx_tokens_election	election_id	Find all tokens for an election (admin dashboard, resend flows)
idx_tokens_is_used	is_used	Find unused tokens for admin resend functionality

blind_tokens

Unlinked ballot authorisation tokens issued after MFA verification. This table has no voter reference of any kind — it cannot be joined to voters or voting_tokens. It exists solely to authorise ballot submission without revealing the submitter's identity.

Column	Type	Constraints	Purpose / Notes
id	SERIAL	PRIMARY KEY	Record identifier — not exposed externally
ballot_token	VARCHAR(255)	UNIQUE NOT NULL	36-character UUID token used to authorise ballot submission
election_id	INTEGER	FK → elections, NOT NULL	Which election this blind token is valid for
is_used	BOOLEAN	DEFAULT FALSE	Whether a ballot has been cast with this token
issued_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the token was issued — after successful MFA
used_at	TIMESTAMP	NULL	When ballot was submitted
NO voter_id	-	CRITICAL	This table intentionally has no voter reference; anonymity enforced at schema level
NO voting_token_id	-	CRITICAL	No link to voting_tokens; prevents identity correlation by any means

5.3.5 Table Group 5 – The Immutable Ledger

encrypted_ballots

The append-only ledger of cast votes. Every ballot is stored encrypted with no reference to voter identity. Two database triggers enforce immutability (no UPDATE or DELETE permitted)

and automatically compute a SHA-256 ballot hash on INSERT.

Column	Type	Constraints	Purpose / Notes
id	SERIAL	PRIMARY KEY	Internal record identifier — not returned to voters
election_id	INTEGER	FK → elections, NOT NULL	Which election — used for vote-counting queries and tally computation
encrypted_vote	BYTEA	NOT NULL	RSA-encrypted AES-256 payload containing the voter's choice
ballot_hash	VARCHAR(255)	NOT NULL	SHA-256 hash auto-generated on INSERT via trigger (includes random UUID for uniqueness)
previous_hash	VARCHAR(255)	NULL	Hash of the previous ballot in sequence — forms the tamper-detection chain
receipt_token	VARCHAR(255)	UNIQUE NOT NULL	Token returned to voter for independent verification — the only external reference to this record
cast_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Server-side submission timestamp; not user-supplied
NO voter_id	–	CRITICAL	No voter reference by design; anonymity enforced
NO blind_token	–	CRITICAL	No blind token reference; prevents any

_id			form of identity linkage
NO user_id	–	CRITICAL	Zero identity encoding; this record cannot be traced to any person

Database triggers on encrypted_ballots:

Trigger	Event	Function	Effect
immutable_ballots	BEFORE UPDATE OR DELETE	prevent_ballot_modification()	Raises exception: 'Encrypted ballots are immutable and cannot be modified or deleted'
auto_ballot_hash	BEFORE INSERT	generate_ballot_hash()	Computes SHA-256(election_id || encrypted_vote || cast_at || random_UUID) and writes ballot_hash and previous_hash

vote_receipts

Public receipt log that allows voters to independently verify their ballot was recorded. Contains only the ballot hash and timestamp — no vote content and no voter identity. Publicly queryable by receipt token.

Column	Type	Constraints	Purpose / Notes
id	SERIAL	PRIMARY KEY	Record identifier

election_id	INTEGER	FK → elections, NOT NULL	Election context for the receipt
receipt_token	VARCHAR(255)	UNIQUE NOT NULL	Matches encrypted_ballots.receipt_token — the voter's verification credential
ballot_hash	VARCHAR(255)	NOT NULL	SHA-256 hash of the ballot — voter compares this against the hash received at submission time
cast_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the ballot was cast

Voter receipt verification flow:

- Voter receives receipt_token from the voting service immediately after submitting their ballot.
- Voter visits: GET /receipt/{receipt_token}
- System returns: ballot_hash, cast_at, election_title; no vote contents and no voter identity.
- Voter compares the returned ballot_hash against the hash they noted at submission time.
- A matching hash confirms the ballot was recorded and has not been tampered with.

audit_log

Append-only ledger of all significant system events. Protected by an immutability trigger. Each entry is SHA-256 hash-chained to the previous entry, enabling forensic verification that no events have been inserted, deleted, or reordered after the fact.

Column	Type	Constraints	Purpose / Notes
id	SERIAL	PRIMARY KEY	Auto-incrementing event identifier
event_type	VARCHAR(50)	NOT NULL	Event category; see Event Types table below

election_id	INTEGER	FK → elections, ON DELETE SET NULL	Election context; NULL for organisation-wide events; preserved as NULL if election is later deleted
actor_type	VARCHAR(20)	CHECK IN ('organiser', 'voter', 'system')	Who performed the action
actor_id	INTEGER	NULL	ID of the actor (organiser_id or voter_id); NULL for system events and voter ballot events (to preserve anonymity)
detail	JSONB	NULL	Structured event detail; e.g. {voter_count: 42, tokens_generated: 42}
event_hash	VARCHAR(255)	NOT NULL	SHA-256 hash auto-generated on INSERT via trigger
previous_hash	VARCHAR(255)	NULL	Hash of the previous audit entry; forms tamper-detection chain
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Event occurrence timestamp

Audit event type reference:

event_type	actor_type	Description	detail JSON Fields
election_created	organiser	Admin created a new election	{election_id, title}
election_opened	organiser	Admin started the voting period	{election_id, voter_count}

election_closed	organiser	Admin closed voting	{election_id, total_votes}
voters_imported	organiser	CSV voter upload completed	{election_id, count, skipped, errors}
tokens_generated	organiser	Voting tokens generated and queued	{election_id, count, expires_days}
token_sent	system	Email delivery attempted or completed	{voter_email_domain, success}
token_validated	voter	Voter clicked their voting link	{election_id}
identity_verified	voter	DOB/MFA check passed	{election_id}
ballot_submitted	voter	Vote successfully cast	{election_id, receipt_token}
double_vote_attempted	voter	Reuse of consumed or used token detected	{election_id, token, reason}
audit_integrity_checked	system	Hash chain verification run	{election_id, status: 'valid' | 'invalid'}

5.4 Business Logic Constraints

Constraint	Implementation	Enforcement Layer
One organiser per email	UNIQUE (email) on organisers	Database; constraint violation on INSERT
One voter per election per email	UNIQUE (election_id, email) on voters	Database; constraint violation on INSERT
One voting token value globally	UNIQUE (token) on voting_tokens	Database; UUID collision probability negligible but enforced

One ballot token value globally	UNIQUE (ballot_token) on blind_tokens	Database; enforced on INSERT
One receipt per vote	UNIQUE (receipt_token) on encrypted_ballots	Database; enforced on INSERT
One vote per voter per election	has_voted flag + transaction isolation	Application + Database; flag set atomically with blind_token issuance
Valid election status values	CHECK (status IN ('draft', 'open', 'closed'))	Database; invalid values rejected on INSERT/UPDATE
Valid organiser roles	CHECK (role IN ('admin', 'super_admin'))	Database; invalid values rejected
Valid audit actor types	CHECK (actor_type IN ('organiser', 'voter', 'system'))	Database; invalid values rejected
Ballots are immutable	BEFORE UPDATE/DELETE trigger on encrypted_ballots	Database trigger; raises exception regardless of DB user or connection method
Audit log is immutable	BEFORE UPDATE/DELETE trigger on audit_log	Database trigger; raises exception regardless of DB user
Election must have >= 2 options to open	Application-level check before status transition	Application code in auth-service
Options cannot be added to open elections	Application-level check on election.status	Application code in election-service

5.5 Security & Data Governance

Data Classification

Table	Classification	Rationale	API Exposure

organisations	Internal	Organisational metadata	Auth-service only; no public API
organisers	Confidential (PII)	Contains email and password hash	Auth-service only; organiser can read own record
elections	Public	Election metadata visible to authenticated users	Read: all authenticated; Write: organiser only
voters	Confidential (PII)	Contains email, date_of_birth, voting status	Auth-service only; no public API; admin read-only
voting_tokens	Confidential (session)	Session credentials; must not leak	Auth-service only
blind_tokens	Internal	Anonymisation bridge must remain unlinked	Auth-service only
encrypted_ballots	Sensitive	Encrypted votes; immutable, no identity	Auth-service only; count via results API only
vote_receipts	Public (anonymous)	Hash and timestamp only;safe to expose	Public via receipt_token lookup
audit_log	Internal (forensic)	Complete event history; admin access only	Auth-service and admin audit tab
voter_mfa	Internal (session)	MFA verification records	Auth-service only

Database User Permissions

Stage 1 – Development Configuration

Stage 1 uses a single privileged database user for all services, appropriate for a development and demonstration environment:

```
CREATE USER voting_admin WITH PASSWORD '...';

GRANT ALL PRIVILEGES ON DATABASE voting_system TO voting_admin;
```

Stage 2 – Recommended Least-Privilege Configuration

Database User	Granted On	Permissions	Notes
auth_service	All tables	SELECT, INSERT, UPDATE (not DELETE on immutable tables)	DELETE on encrypted_ballots and audit_log blocked by trigger as defence-in-depth
analyst (read-only)	audit_log, elections, election_options, tallied_votes	SELECT only	Denied SELECT on voters, voting_tokens, organisers
migration_runner	All tables	ALL (during migration window only)	Permissions applied during deployment window and revoked immediately after

Vote Immutability

Two triggers; prevent_vote_update and prevent_vote_delete, fire BEFORE UPDATE and BEFORE DELETE operations on the votes table. Both raise a PostgreSQL exception, making the trigger enforcement independent of application code. Even a direct psql connection with the voting_service database user cannot modify vote records.

Audit Log Immutability

The audit_immutability trigger fires BEFORE DELETE OR UPDATE on the audit_logs table and raises an exception, guaranteeing that all audit records are permanent and unmodifiable regardless of which database user makes the request.

6. Technology Selection & Rationale

6.1 Backend Framework – FastAPI (Python)

The UVote system requires a backend framework capable of handling concurrent voter submissions during active elections. The project initially prototyped with Flask. Flask's synchronous WSGI model is a documented limitation under concurrent load, as blocking database calls cause significant response time degradation. This ruled out Flask as a production option and prompted evaluation of natively asynchronous alternatives.

The team's primary language is Python (3+ years of experience). Switching to a different language for the backend would consume 4 to 6 weeks of the 12-week Stage 1 timeline, making a Python-based solution a firm constraint.

The selected approach is FastAPI, an ASGI framework built on Starlette and Pydantic. It provides native async/await support, type-hint-based request validation, and automatic OpenAPI documentation generation. All six backend microservices will follow a consistent FastAPI application structure, sharing a common asyncpg connection pool and security module.

All five U-Vote microservices are implemented using FastAPI. Each service follows the same application structure: a FastAPI app instance, startup/shutdown lifecycle hooks connecting to the shared asyncpg pool, and a /health endpoint.

A representative service entrypoint:

```
from fastapi import fastapi

from shared.database import database

app = fastapi(title="auth service", version="1.0.0")

@app.on_event("startup")

async def startup():

    await database.get_pool()

@app.on_event("shutdown")

async def shutdown():

    await database.close()

@app.get("/health")

```

```
async def health():

    return {"status": "healthy", "service": "auth"}
```

Alternative Considerations

Flask (rejected): Flask was the most familiar option given DkIT coursework. It is unsuitable for this use case because its synchronous WSGI model blocks on database calls under concurrent load. Flask-Async exists but is not considered production-grade. Workarounds using gevent add complexity without matching the reliability of a natively asynchronous framework. Flask also requires separate libraries for JWT, input validation, and OpenAPI documentation, whereas FastAPI includes these capabilities.

Node.js with Express (rejected): Express provides native async I/O via JavaScript's event loop and would meet the performance requirements. It was rejected because all infrastructure scripts, shared libraries, and DevOps tooling are written in Python. Introducing a second language increases cognitive overhead without a proportional benefit at the scale of this project.

Go with Gin (rejected): Gin would provide the best raw performance of the options considered. It was rejected because Go is entirely new to the team, and the estimated learning investment of 4 to 6 weeks is not compatible with the Stage 1 timeline. The performance difference between Go and FastAPI is not meaningful for an election system with a target of 1,000 concurrent voters.

Java with Spring Boot (rejected): Spring Boot was rejected due to JVM startup times of 10 to 30 seconds, a baseline memory footprint of approximately 200 MB per service, and the overhead of Java boilerplate. These characteristics are poorly suited to lightweight microservices running in a resource-constrained Kind cluster.

Pydantic v2 compatibility: Some third-party libraries were slow to adopt Pydantic v2 following its 2024 release. This is mitigated by pinning the Pydantic version in requirements.txt for each service.

Learning curve: The team had little to no prior exposure to FastAPI. The estimated onboarding time was one week, based on Python familiarity and the quality of FastAPI's official documentation. This was accepted as a reasonable trade-off given the performance gains.

6.2 Database – PostgreSQL 15

The U-Vote system requires a database that can guarantee ballot integrity through ACID transactions, provide native encryption primitives for ballot confidentiality, and enforce audit log immutability at the database level. The database must also support per-service access

control to align with the microservices architecture, where each service should only access the tables relevant to its function.

The election domain has strict data integrity requirements. A lost or duplicated vote is a critical failure. Unlike an e-commerce system where a retry is acceptable, a double-counted ballot undermines the legitimacy of an entire election.

The selected approach was PostgreSQL 15, an open-source relational database with native support for all identified requirements via the pgcrypto extension, PL/pgSQL triggers, and a granular role-based access control system.

Connection pool configuration (shared/database.py):

```
import asyncpg

class Database:

    _pool = None

    @classmethod

    async def get_pool(cls):

        if cls._pool is None:

            cls._pool = await asyncpg.create_pool(

                host=os.getenv("DB_HOST", "postgres-service"),

                port=int(os.getenv("DB_PORT", 5432)),

                database=os.getenv("DB_NAME", "uvote"),

                user=os.getenv("DB_USER"),

                password=os.getenv("DB_PASSWORD"),

                min_size=2,

                max_size=20,

                command_timeout=30

            )

        return cls._pool
```

Audit log immutability trigger (schema.sql):

```
CREATE OR REPLACE FUNCTION prevent_audit_modification()

RETURNS TRIGGER AS $$

BEGIN

    RAISE EXCEPTION 'Audit log records cannot be modified or deleted';

    RETURN NULL;

END;

```

```
$$ LANGUAGE plpgsql;

CREATE TRIGGER audit_immutability

BEFORE DELETE OR UPDATE ON audit_log

FOR EACH ROW

EXECUTE FUNCTION prevent_audit_modification();
```

Ballot encryption (schema.sql):

```
INSERT INTO ballots (election_id, voter_token_hash, encrypted_ballot)

VALUES (

    $1,

    $2,

    pgp_sym_encrypt($3::text, current_setting('app.encryption_key'))

);


```

Alternate Considerations:

MySQL 8.0 (rejected): MySQL is familiar from DkIT coursework and provides ACID compliance via the InnoDB engine. It was rejected primarily because it has no equivalent to pgcrypto for in-database encryption. MySQL's AES_ENCRYPT function exists but lacks PGP-grade primitives, meaning ballot encryption would have to be handled entirely at the application layer. MySQL's trigger system is also less expressive than PL/pgSQL for audit enforcement, and its per-user privilege model is less granular than PostgreSQL's role-based system. The asyncmy async driver is less mature and performant than asyncpg.

MongoDB 7.0 (rejected): MongoDB provides flexible schema and native document storage. It was rejected on three grounds. First, multi-document ACID transactions were added in v4.0 but carry a documented performance penalty. Second, in-database encryption requires MongoDB Enterprise, which is a paid product. Third, change streams are reactive rather than preventive and cannot enforce audit immutability in the same way as a PL/pgSQL trigger. More fundamentally, the election data model is relational by nature: voters are registered for elections, ballots reference candidates, and results aggregate votes. Representing this in a document model adds complexity without benefit.

SQLite 3 (rejected): SQLite was used for early prototyping due to its zero-configuration setup. It is unsuitable for production use in this system because its file-level locking permits only one writer at a time, which is incompatible with concurrent ballot submissions from multiple services. SQLite also has no user or role system, no native encryption extension in its standard build, and no support for asyncpg. It is not suitable for multi-service access across containers.

Operational complexity: PostgreSQL requires server process management, user creation, and extension installation. This is accepted as a one-time setup cost given the security requirements it satisfies.

Single point of failure: One PostgreSQL instance serves all six microservices. If the database becomes unavailable, all services are affected. This is mitigated by Kubernetes health probes and PVC-backed storage, which ensures data survives pod restarts. A multi-node setup is out of scope for this project.

Key management: pgcrypto's symmetric encryption requires the encryption key to be distributed securely. The key is stored as a Kubernetes Secret and injected as an environment variable. A compromise of the Secret would compromise ballot confidentiality. This is an accepted risk within the project's scope.

Decision

PostgreSQL 15 was selected as the database for all U-Vote microservices.

PostgreSQL is the only option evaluated that satisfies all security-critical requirements: native ballot encryption via pgcrypto, audit log immutability via PL/pgSQL triggers, and per-service user roles with least-privilege grants. These controls are enforced at the database layer, which provides a stronger security posture than application-level enforcement alone. A compromised service cannot bypass database-level triggers or exceed its granted privileges.

The three security factors were the primary drivers. The pgp_sym_encrypt function encrypts ballot content within the database using AES-256, meaning ballot plaintext is not persisted in unencrypted form. The BEFORE DELETE OR UPDATE trigger on the audit_log table raises an exception for any attempt to modify or delete audit records, making the log append-only regardless of which service is making the request. Per-service database users are granted only the privileges required for their function: the voting service can insert ballots but cannot read them, and the results service can read and decrypt ballots but cannot modify them.

JSONB columns with GIN indexing allow election configuration to evolve without schema migrations, which suits iterative development.

6.3 Container Orchestration – Kubernetes (Kind)

Docker Compose was used during initial development for local multi-service orchestration. While adequate for starting containers together, Compose lacks the production-grade features required by the module brief: network policies for service isolation, health-based traffic routing, rolling update strategies, and namespace-based environment separation. The module assessment rubric specifically allocates marks for provisioned infrastructure using Infrastructure as Code and resilience evidence, both of which require an orchestration platform beyond Docker Compose.

The selected approach is Kubernetes, deployed locally using the Kind (Kubernetes IN Docker) distribution. All cluster configuration is version-controlled as YAML manifests, and the Calico CNI is used for network policy enforcement.

Proposed Design

Services and modules impacted:

All six UVote microservices are deployed as Kubernetes Deployments within the uvote-dev, uvote-test, and uvote-prod namespaces. Each service is configured with liveness and readiness probes, resource limits, and a minimum of 2 replicas for high availability.

Cluster topology:

- Distribution: Kind (see ADR-012 for distribution selection rationale)
- Nodes: 1 control-plane, 2 workers
- CNI: Calico v3.26.1 (default CNI disabled)
- Namespaces: uvote-dev, uvote-test, uvote-prod
- Storage: PersistentVolumeClaim (5Gi) for PostgreSQL
- Ingress: Nginx Ingress Controller via Helm

Kind cluster configuration (kind-config.yaml):

```yaml
kind: Cluster

apiVersion: kind.x-k8s.io/v1alpha4

nodes:

  - role: control-plane

  - role: worker

  - role: worker

networking:

  disableDefaultCNI: true  # Required for Calico

  podSubnet: "192.168.0.0/16"  # Calico's default CIDR
```

Considerations

Docker Swarm (rejected): Docker Swarm provides basic clustering and service replication using familiar Docker CLI commands. It was rejected because it has no NetworkPolicy support, which is a hard requirement for the zero-trust security model. Swarm also lacks readiness probes, namespace isolation, and the rolling update strategies required by the

module brief. Its industry adoption has declined as Docker Inc. shifted focus to Docker Desktop.

Docker Compose only (rejected): Docker Compose is already in use for local development and has near-zero learning curve. It was rejected as a production platform because it provides no replicas, no health probes, no rolling updates, no network policies, no namespace isolation, and no secret management beyond plaintext values in YAML or .env files. It does not satisfy the module requirement for a cloud-native operational platform.

HashiCorp Nomad (rejected): Nomad is a lighter-weight orchestrator that can schedule containers alongside other workload types. It was rejected because it has no native NetworkPolicy equivalent (service mesh via Consul Connect would be required), it has significantly less industry adoption than Kubernetes, and it requires learning HCL configuration in addition to its own concepts. The smaller ecosystem means fewer ready-made solutions for monitoring, logging, and CI/CD integration.

Why Kubernetes Was Selected

Kubernetes was selected as the container orchestration platform for UVote.

Kubernetes is the only option evaluated that satisfies all requirements. The primary driver is module alignment: the PROJ I8009 brief explicitly requires an operational platform on cloud-native technologies, and Kubernetes is the canonical implementation of this requirement. The module assessment rubric allocates marks for Infrastructure as Code and resilience evidence, both of which are first-class Kubernetes features.

The second driver is network policy enforcement. The zero-trust security model (ADR-010) requires NetworkPolicies enforced by a CNI plugin. Of the options considered, only Kubernetes provides this capability natively. Calico is selected as the CNI for its mature NetworkPolicy support and compatibility with Kind.

The third driver is industry relevance. According to the CNCF Survey 2023, 96% of organisations adopting container orchestration use Kubernetes. Demonstrating Kubernetes competency is directly relevant for employment after graduation.

Replicas, health probes, rolling updates, Secrets, and namespaces are all first-class Kubernetes features that require no workarounds or third-party additions to satisfy the project requirements.

The trade-off of increased operational complexity and higher resource usage over Docker Compose is accepted. Compose is a development tool and is fundamentally inadequate as an operational platform for the project's requirements. The same Kubernetes manifests, with

minor modifications, can be used to deploy UVote to a cloud Kubernetes service such as AKS or EKS.

6.4 CNI Plugin - Calico v3.26.1

Calico was selected as the Kubernetes Container Network Interface (CNI) plugin.

Why Calico Was Selected

- Kubernetes does not enforce NetworkPolicies itself — enforcement is delegated to the CNI plugin. Without a CNI that supports enforcement, all NetworkPolicy resources are accepted but have no effect.
- Calico provides full enforcement of the Kubernetes NetworkPolicy API including ingress and egress rules, label-based pod selection, and port-level filtering — the complete feature set required by the zero-trust security model.
- Calico has well-documented compatibility with Kind clusters and was recommended by the DkIT DevOps module lecturer.
- Calico is used by major cloud providers including Azure AKS and Amazon EKS, meaning the configuration knowledge transfers directly to future cloud deployments.

Considerations

Cilium (not selected): Cilium is a modern CNI using eBPF for networking, security, and observability. It provides full NetworkPolicy support and adds L7-aware policies that can filter by HTTP path and headers. It was not selected for three reasons. First, it requires a kernel version of 5.4 or higher for its full eBPF feature set, which adds an environmental dependency. Second, its resource overhead is higher than Calico at approximately 400MB for system pods. Third, its Kind compatibility requires additional configuration steps that are less well-documented than Calico's. The L7 filtering capabilities are also not needed for this project, as all UVote network policies operate at L3/L4 only.

Flannel (not selected): Flannel is a simple overlay network that provides basic pod-to-pod connectivity. It was not selected because it has no NetworkPolicy support. Policies are accepted by the API server but silently ignored, meaning a default-deny policy has no effect. This is a hard disqualifier given the project's zero-trust security requirement.

Weave Net (not selected): Weave Net provides mesh networking with optional built-in traffic encryption and basic NetworkPolicy support. It was eliminated because Weaveworks, the company behind it, ceased operations in February 2024. With no active maintainer, it is not a suitable dependency for a project continuing into 2026.

Standard API usage: All UVote network policies use the standard networking.k8s.io/v1 API rather than Calico-specific CRDs. This is an intentional design choice that keeps policies portable to any compliant CNI, including Cilium if a migration is needed in Stage 2.

6.5 Additional Technology Choices

Component	Technology	Rationale
Frontend rendering	Jinja2 (server-side)	Simpler deployment than a separate SPA framework; eliminates CORS complexity; suits server-rendered admin and voter UIs
Async DB driver	asyncpg	Native async PostgreSQL driver with the best performance characteristics for FastAPI; no ORM overhead
Password hashing	bcrypt (cost 12)	Industry standard for password storage; cost factor 12 provides strong resistance to brute-force attacks
JWT library	PyJWT	Lightweight, well-maintained; integrates natively with FastAPI dependency injection without wrappers
Ingress controller	Nginx (Helm)	Mature, well-documented; Helm chart simplifies configuration; widely supported in Kubernetes ecosystems
Cluster distribution	Kind	Simplest local Kubernetes distribution; runs inside Docker; ideal for development and CI without a cloud provider

7. Security Design

7.1 Admin Authentication

Admin passwords are stored exclusively as bcrypt hashes with a cost factor of 12. The plaintext password is discarded immediately after hashing and is never written to any persistent store or log. On login, the submitted password is compared against the stored hash using constant-time comparison to prevent timing attacks.

JWT tokens are signed using the HS256 algorithm with a secret key stored as a Kubernetes Secret. Tokens carry a 24-hour expiration and include the admin's role claim, which is validated on each protected request. The API gateway validates the JWT signature before forwarding any request to a backend service, so no service is reachable without a valid token.

Account lockout is enforced after five consecutive failed login attempts. The login endpoint is additionally rate-limited at the gateway level.

7.2 Voter Authentication

Voters are not required to create accounts or set passwords. Authentication is achieved through one-time cryptographic tokens generated using Python's secrets.token_urlsafe(32), which produces 256 bits of entropy from the operating system's cryptographically secure random source. Each token is 43 characters long, URL-safe, and statistically unique.

Tokens are stored in the voting_tokens table with a 7-day expiry and are invalidated immediately upon use. The is_used flag is set to TRUE and used_at is recorded within the same database transaction as the vote insertion, preventing race conditions.

7.3 Vote Anonymity

The votes table contains no voter_id column. It stores only the election_id, candidate_id, vote_hash, previous_hash, and cast_at timestamp. The link between a voter and their token is severed at the point of vote submission: the token record is updated (is_used = TRUE) but no reference to the token or voter is written to the votes table.

The audit log records that voter X cast a vote in election Y, but does not record which candidate they chose. There is no database query that can reconstruct which candidate a specific voter selected after the vote has been cast.

7.4 Data Integrity

The votes table is protected by two database-level triggers (prevent_vote_update and prevent_vote_delete) that raise PostgreSQL exceptions on any modification or deletion attempt. These triggers are enforced at the database layer — application code cannot bypass them even with a direct database connection.

Vote records are hash-chained using SHA-256, linking each vote to the previous one for the same election. Any attempt to insert, reorder, or remove a vote record breaks the chain and is

detectable during a verification pass. Audit logs are similarly hash-chained and protected by an immutability trigger.

7.5 Network Security

The cluster uses a zero-trust network security model enforced by Calico NetworkPolicies. The default posture is deny-all: no pod-to-pod communication is permitted unless explicitly allowed by a NetworkPolicy resource. Services cannot reach each other directly, all inter-service traffic is routed via the API gateway. The PostgreSQL port (5432) is exposed only within the cluster and is not accessible from outside.

7.6 Secret Management

Database credentials, the JWT signing key, and the ballot encryption key are stored as Kubernetes Secrets and injected into pods as environment variables at runtime. Secrets are never committed to version control. Each service reads only the secrets it requires.

7.7 Input Validation

All HTTP request inputs are validated by Pydantic models before reaching business logic. Pydantic v2 performs strict type checking, length validation, and format validation (including email format). SQL queries throughout the codebase use parameterised statements exclusively — no string interpolation is used in SQL, preventing SQL injection.

7.8 Planned Security Enhancements (Stage 2)

- HTTPS certificate management using cert-manager with Let's Encrypt for production deployments.
- RBAC refinement to restrict which admins can manage which elections (multi-tenancy).
- Optional voter password authentication for vote receipt verification (hybrid token + password model).
- Rate limiting on all API endpoints (currently applied to the login endpoint only).

8 Design Decisions, Investigations & Iterations

8.1 Iteration 1 – Prototyping with Flask and SQLite

The first development iteration used Flask and SQLite to rapidly prototype the core voting flow. This allowed the team to validate the token-based voter authentication concept and the basic election CRUD operations without infrastructure overhead.

The limitation of this approach became apparent during load testing: Flask's synchronous WSGI model blocks on every database call. Under concurrent voter load, response times degraded significantly as the thread pool exhausted. SQLite's file-level write locking compounded this — only one write operation could proceed at a time, making concurrent vote submissions queue serially.

Outcome: Flask and SQLite were ruled out as production technologies. The prototype confirmed the data model and the voting flow were sound, and the code was refactored to FastAPI and asyncpg in the next iteration.

8.2 Iteration 2 – FastAPI and PostgreSQL

The second iteration introduced FastAPI with asyncpg and PostgreSQL 15. The shared database.py connection pool module was written to provide a singleton asyncpg pool used by all services. The schema.sql file was written to define the full table structure, indexes, triggers, and per-service user grants.

The pgcrypto extension was investigated as the mechanism for ballot encryption. The pgp_sym_encrypt function was tested and confirmed to store AES-256-encrypted ballot content within the database. The audit log trigger was implemented and validated by attempting UPDATE and DELETE on the audit_logs table from a psql session.

Outcome: The FastAPI and PostgreSQL layer was confirmed to be production-ready. The test_db.py test suite was written in this iteration to validate schema correctness, trigger behaviour, and database role permissions.

8.3 Iteration 3 — Kubernetes Platform

The third iteration introduced the Kubernetes platform. The team evaluated Kind as the local Kubernetes distribution and created the kind-config.yaml with the default CNI disabled to allow Calico installation. The setup_k8s_platform.py provisioning script was written to automate cluster creation, Calico installation, namespace creation, database deployment, schema application, and network policy installation.

The decision to use Python scripts rather than Bash for all automation was deliberate: the team is split across Linux and macOS, and Python runs without modification on both

platforms. Python's subprocess module provides the same capability as shell scripting while avoiding OS-specific shell compatibility issues.

Outcome: The Kubernetes platform was successfully provisioned using the automated script. The deploy_platform.py script was written to build Docker images, load them into Kind, create Kubernetes Secrets, and deploy all six services.

8.4 Technology Evaluation Evidence

The following evaluations were conducted and documented during Stage 1:

- FastAPI vs Flask: Benchmarked concurrent request handling. FastAPI with asyncpg achieved over 1,000 requests/second under load; Flask with psycopg2 showed blocking degradation under concurrent load.
- PostgreSQL vs MySQL: Evaluated pgcrypto (PostgreSQL) against AES_ENCRYPT (MySQL). Concluded pgcrypto provides PGP-grade encryption primitives that MySQL does not.
- Calico vs Flannel: Confirmed that Flannel does not enforce NetworkPolicy resources - policies are accepted by the API server but have no effect. This disqualified Flannel for the zero-trust model.
- Weave Net eliminated: Confirmed that Weaveworks ceased operations in February 2024, making Weave Net unsuitable as a long-term dependency.

9. Reliability, Scalability & High Availability

9.1 Replicas and Pod Distribution

Each of the six UVote microservices is deployed with a minimum of two replicas in the Kubernetes cluster. Replicas are distributed across the two worker nodes by the Kubernetes scheduler, ensuring that a single node failure does not take an entire service offline. The Kubernetes Service resource provides automatic load balancing across healthy replicas using round-robin selection.

9.2 Health Probes

Every microservice exposes a /health endpoint that returns HTTP 200 when the service is ready to accept traffic. Kubernetes liveness probes restart pods that fail health checks, and readiness probes gate traffic routing — a pod that is not yet ready will not receive requests from the Service load balancer. The FastAPI startup lifecycle hook establishes the asyncpg

connection pool before the readiness probe is satisfied, ensuring that database connectivity is confirmed before traffic is routed.

9.3 Rolling Updates

Kubernetes Deployment manifests use the RollingUpdate strategy, which replaces old pods with new ones incrementally. At no point during a deployment are all replicas simultaneously replaced — at least one replica of each service remains available throughout. Combined with readiness probes, this ensures zero-downtime deployments for routine updates.

9.4 Database Durability

PostgreSQL data is stored on a Kubernetes PersistentVolumeClaim of 5Gi with ReadWriteOnce access mode. The PVC is backed by the underlying host storage and survives pod restarts and rescheduling. If the PostgreSQL pod is killed, Kubernetes will restart it and it will re-attach to the same PVC, recovering all data without intervention.

9.5 Database Connection Pooling

The shared database.py asyncpg connection pool is configured with a minimum pool size of 2 connections and a maximum of 20 connections per service replica. Under peak load, each replica can maintain up to 20 concurrent database connections. With two replicas, each service can sustain 40 concurrent database connections. The command timeout of 30 seconds prevents slow queries from holding connections indefinitely.

9.6 Scalability Considerations

The current architecture runs all services in a single Kind cluster on a single development machine. For production scaling, the same Kubernetes manifests can be applied to a cloud Kubernetes service (AKS, EKS, GKE) with minimal modification. Each service can be scaled independently — the Voting Service can be scaled to more replicas during an active election without scaling services that are idle.

The primary scaling constraint is the single PostgreSQL instance. Under very high concurrent write load, the database could become a bottleneck. PgBouncer connection pooling and read replicas are identified as Stage 2 improvements if load testing reveals this constraint.

9.7 Known Single Points of Failure (Current Stage)

Component	Current Status	Planned Mitigation
PostgreSQL	Single instance, if the pod is unavailable, all services are affected	PgBouncer + read replicas in Stage 2; Kubernetes health probes mitigate transient failures
Nginx Ingress	Single controller instance	Ingress HA configuration in Stage 2 cloud deployment
Email Service	External SMTP dependency	Retry logic implemented; SMTP provider redundancy in Stage 2

10. Test & Evaluation of Prototype Functionality

10.1 Testing Strategy Overview

Stage 1 testing is structured across three scripts that validate successive layers of the platform. Each script is a prerequisite for the next, and all are written in Python to ensure cross-platform compatibility (Linux and macOS).

Script	Purpose	Layer Validated
setup_k8s_platform.py	Provisions the Kind cluster, installs Calico, deploys PostgreSQL, applies schema, and installs network policies	Infrastructure
tests/test_db.py	Validates the deployed PostgreSQL database, schema, seed data, security controls, triggers, indexes, and network connectivity	Database
plat_scripts/deploy_platform.py	Builds Docker images, loads them into Kind, deploys services, and runs health endpoint smoke tests	Application

10.2 Database Test Suite (test_db.py)

The test_db.py suite executes 12 tests covering five categories. All tests run SQL via kubectl exec into the PostgreSQL pod. Test 1 acts as a gate — if the PostgreSQL pod is not in Running state with 1/1 containers ready, the entire suite exits without running further tests.

Test #	Name	What it Validates	Category
1	test_pod_running	PostgreSQL pod is Running with 1/1 containers ready	Infrastructure
2	test_connection	SELECT version() returns a row containing 'PostgreSQL'	Infrastructure
3	test_tables_exist	All 7 tables present: admins, elections, candidates, voters, voting_tokens, votes, audit_logs	Schema
4	test_sample_data	admins, elections, and candidates each contain at least 1 seed row	Data
5	test_vote_immutability	UPDATE and DELETE on votes both raise the trigger exception; INSERT succeeds	Security
6	test_hash_generation	vote_hash is non-NULL in at least one of the last 5 votes after INSERT	Security
7	test_user_permissions	auth_service can SELECT admins but is denied on votes; results_service can SELECT votes but is denied on INSERT	Security
8	test_complex_queries	Multi-table JOIN across elections, candidates, and	Functionality

		votes returns rows with a candidate column	
9	test_indexes	At least one of the critical indexes (idx_votes_election, idx_votes_candidate, idx_tokens_token) exists	Performance
10	test_foreign_keys	Foreign key constraint count is greater than 0 in information_schema	Schema
11	test_connection_from_pod	A temporary pod with the auth-service label can connect to postgresql:5432 via ClusterIP	Network
12	test_load_performance	Bulk INSERT of N votes succeeds; distribution query returns rows (disabled by default)	Performance

10.3 Platform Deployment Tests (deploy_platform.py)

The deploy_platform.py script executes 9 sequential phases, each validating a different layer of the deployment:

Phase	What it Tests
1. Pre-flight checks	Verifies Kind cluster is running, Docker daemon is available, target namespace exists, and PostgreSQL pod is ready
1. Image build	Builds all 6 service Docker images with a 600-second timeout per build
1. Image load	Loads all 6 images into the Kind cluster using kind load docker-image

1. Secrets	Creates or updates db-credentials, jwt-secret, and flask-secret Kubernetes Secrets; syncs the PostgreSQL password
1. Service manifests	Applies all service Deployment and Service manifests from the uvote-platform/k8s/services/ directory
1. Pod readiness	Polls pod readiness every 5 seconds up to the configured timeout (default 300 seconds)
1. TCP connectivity	Tests TCP connectivity from each backend service pod to postgresql:5432; verifies that the frontend pod connection is blocked by NetworkPolicy
1. Health endpoints	Opens a port-forward tunnel per service and checks that the /health endpoint returns HTTP 200
1. Summary	Prints pass/fail counts for all phases and declares an overall pass or fail result

10.4 Known Test Limitations

- Test script ordering is not enforced programmatically — running test_db.py or deploy_platform.py against an incomplete cluster produces failures that resemble test failures rather than setup failures.
- Calico is pulled from Docker Hub at runtime — a restricted network will cause Step 2 of setup_k8s_platform.py to fail or stall with no offline fallback.
- Test 7 in test_db.py covers only two of the six database roles. election_service, voting_service, and admin_service permission verification is not yet included.
- Test 12 (load performance) is disabled by default and should not be run on a machine in active use — it has caused development laptop instability. Votes inserted by this test are permanent due to the immutability trigger.

10.5 Functional Prototype Evaluation

The following functional capabilities were confirmed as working in the Stage 1 prototype:

- Admin registration, login, and JWT authentication via the Auth Service.
- Election creation, activation, and closure via the Election Service lifecycle state machine.
- Candidate addition and removal via the Admin Service.

- Voter import via CSV upload and manual entry via the Admin Service.
- Voting token generation using secrets.token_urlsafe(32) with 7-day expiry.
- Voting invitation email dispatch via the Email Service.
- Token validation on voter URL click – expired and used token scenarios handled correctly.
- Vote submission with anonymous storage and database-level hash chaining.
- Double-vote prevention enforced by the UNIQUE(voter_id, election_id) constraint on voting_tokens.
- Vote immutability enforced by database triggers.
- Results tallying, percentage calculation, and winner determination via the Results Service.
- WCAG AA compliant templates on all voter-facing pages.

11 Conclusion and Stage 2 Enhancements

The system successfully implements identity-ballot separation, immutable hash-chained audit logs, microservices architecture, and zero-trust network policies via Calico.

Recommended Enhancements

- **MFA for Admins**: TOTP (pyotp) + WebAuthn support for organizers
- **OTP Phone Verification for Voters**: SMS-based second factor (Twilio/AWS SNS) after token validation in place of DOB as well as additional security questions
- **Redis Caching**: token lookup optimization to reduce DB load during high-volume voting
- **Apache Kafka Event Bus**: decouple voting, audit, and email services via topics (token.generated`, vote.cast, audit.event, email.request) for improved resilience and scalability
- **External Email Service**: AWS SES/SendGrid integration to replace local SMTP (via email-service consuming Kafka topic)
- **MongoDB for Audit Archival**: replicate historical audit logs to MongoDB asynchronously (keep PostgreSQL as primary for transactional voting data)
- **KMS Integration**: migrate admin JWT from HS256 (symmetric) to asymmetric RSA/EdDSA signing via AWS KMS or HashiCorp Vault
- **Log Hashing**: hierarchical hash commitments to prevent tampering even if database is compromised

Why these choices

- **Kafka**: decouples services, enables replaying events, supports 10k+ concurrent voters without bottlenecks
- **MongoDB**: optional for audit overflow only; PostgreSQL remains primary because ACID and relational constraints enforce single-use tokens

- **KMS + Asymmetric JWT**: eliminates risk of shared secret compromise; enables key rotation without service restarts
- **SMS OTP + External Email**: adds second authentication factor, removes SMTP infrastructure burden, provides delivery guarantees

12 AI Provenance

Use of AI Tools

This project made use of GitHub Copilot and Claude AI to accelerate development. AI assistance was used only as a productivity tool to implement patterns and code the team understood, not to generate novel architectures or bypass learning.

AI-Assisted Components

Component	AI Tool	Scope	Team Understanding
Pydantic v2 schema definitions (shared/schemas.py)	GitHub Copilot	Generated type hints and validators based on patterns shown	Full: team manually reviewed and validated all schemas
Database migration triggers (SQL in init.sql)	Claude	Generated PL/pgSQL audit log immutability and vote hash-chaining triggers	Full: team tested triggers with UPDATE/DELETE attempts; confirmed exceptions raised
FastAPI endpoint templates (auth-service/app.py)	GitHub Copilot	Generated POST /login, POST /verify endpoint skeletons	Full: team implemented authentication flow, JWT signing, and password hashing
Kubernetes manifests (k8s/*.yaml)	Claude	Generated Deployment, Service, NetworkPolicy structure	Full: team defined all resource requirements,

			probes, and network policies
Email utility (shared/email_util.py)	GitHub Copilot	Generated aiosmtplib connection and Jinja2 template rendering	Full: team implemented SMTP configuration, retry logic, and TLS mode handling
Unit test suite (tests/test_db.py)	Claude	Generated test structure and SQL queries for database validation	Full: team wrote all assertions and integrated into provisioning pipeline

AI Limitations & Manual Work

The following aspects were implemented entirely by the team without AI assistance:

- **Architecture decisions:** Microservice decomposition, service responsibilities, inter-service communication patterns
- **Development:** Login and Registration (bycrypt, JWT), as well as email delivery (SMTP retry logic)
- **Security model:** Zero-trust network policies, per-service database user grants, ballot anonymity enforcement
- **Testing strategy:** Test harness design, provisioning script orchestration, load testing approach
- **DevOps automation:** Kind cluster setup, Calico CNI installation, Docker image building and loading
- **Troubleshooting:** SMTP egress blocking diagnosis, Docker network debugging, Kubernetes probe tuning

Code Review & Understanding

All code generated with AI assistance was:

1. Manually reviewed by the team member who requested it
2. Tested with unit tests or manual testing before committing
3. Modified where necessary to align with the project architecture

4. Fully understood by the team, not treated as a "black box"

Example: The vote hash-chaining trigger generated by Claude was tested by attempting an UPDATE on the votes table and confirming that the trigger exception was raised. The team understands exactly how the chain link works and could modify it if needed.

Rationale & Justification

AI was used to accelerate implementation of well-understood patterns (FastAPI endpoints, Kubernetes resources, SQL triggers) where the pattern is standard and the learning curve is steep. This allowed the team to:

- Focus human effort on architectural decisions and testing
- Avoid learning years of template syntax for each framework
- Deliver a more complete prototype within the 12-week Stage 1 timeline
- Demonstrate cloud-native and DevOps practices rather than spending time on boilerplate

AI was NOT used to:

- Design the voting flow or security model
- Select technologies or make architectural trade-offs
- Write the core business logic (token generation, vote casting, anonymity enforcement)
- Generate documentation content (only structure)